PROJECT REVIEW GUIDE · BEGINNER FRIENDLY

CSE-Tutor 1.5B

How we taught a small AI model to tutor six Computer Science subjects, step by step, with training data whose answers are checked by code.

DSA · Programming · DBMS · Operating Systems · Computer Networks · Deep Learning

9,335
examples in the main training mix (plus a 2,107-example DL stage)

3,000
problems with code-verified answers

50.5 → 60.0
Spider SQL accuracy, base → fine-tuned

1 laptop
everything trained on an integrated GPU

Satya Dedeepya · Deep Learning course project · Code: github.com/satyadedeepya/slm_dl_project
Base model: Qwen2.5-1.5B-Instruct · Method: LoRA fine-tuning · Hardware: Intel Core Ultra 5 laptop (Arc iGPU)

START HERE

The whole project on one page

> **IN ONE SENTENCE**
> We took a small, free AI model that runs on a laptop, taught it Computer Science using carefully checked training data, and then measured fairly whether it actually got better.

The problem

Students need help with CS subjects, especially **numerical problems** like CPU scheduling, page faults, subnetting, normal forms and CNN output sizes. Big AI models (like ChatGPT) are good at this but need the internet and can cost money. Small models run on your own laptop for free, but they get these numerical problems wrong.

What we did, in five steps

1. Baseline
Measured how good the untrained model is.

2. Data
Built 9,335 examples, including 2,400 code-verified problems.

3. Train
Fine-tuned with LoRA: FT-B (full) and FT-A (without verified data).

4. Analyse
Tested on unseen questions, checked statistics, studied 50 mistakes.

5. Extend
Added Deep Learning (FT-C), a chat app and a research paper.

What we found

- **Fine-tuning worked where the data was strong:** SQL writing went from 50.5% to 60.0%, database (FD) problems from 15% to 30%, networking numericals from 1% to 8%. All three are statistically significant.
- **The verified data is the reason.** A copy of the model trained without it (FT-A) did not improve on these problems at all.
- **Nothing broke:** coding ability (MBPP) stayed the same.
- **Deep Learning was added in about 70 minutes of training** without losing earlier skills; the share of correct DL numbers rose from 15.8% to 26.0%.
- **Honest weakness:** the model learns the *method* but still makes calculation and step-tracking mistakes in long problems.

Contents

1. Basic ideas you need (AI model, fine-tuning, LoRA, evaluation)
2. What is new in this project (novelty)
3. Step 1: Planning and the baseline
4. Step 2: Building the dataset
5. Step 3: Training on a laptop
6. Step 4: Results, explained
7. Step 5: Error analysis
8. Step 6: Adding Deep Learning
9. Step 7: The chat app, demo and paper
10. How to run everything · Viva questions · Glossary

CHAPTER 1

Basic ideas you need

What is a language model?

A language model is a program that has read a huge amount of text and learned to predict the next word. When you ask it a question, it writes an answer one word (token) at a time. ChatGPT and Claude are **large** language models with hundreds of billions of parameters.

A **parameter** is one number inside the model that was learned during training. More parameters usually means more knowledge, but also more memory and a more expensive computer.

What is a small language model (SLM)?

We used **Qwen2.5-1.5B-Instruct**, which has 1.5 billion parameters. That is small enough to run on a normal laptop with no internet and no cost. The catch: out of the box it scored only 4% on OS numericals and 1% on CN numericals in our tests.

What is fine-tuning?

> ANALOGY
> The base model is like a student who has read a lot of general books. Fine-tuning is like giving that student a focused **CSE workbook with worked solutions**, so they learn the exact style and steps examiners expect.

Technically, we show the model thousands of (question, ideal answer) pairs and slightly adjust its parameters so that it becomes more likely to produce the ideal answers. This is called **supervised fine-tuning (SFT)**.

What is LoRA, and why use it?

Changing all 1.5 billion parameters needs a lot of memory. **LoRA (Low-Rank Adaptation)** freezes the original model and adds small "adapter" matrices next to its layers. Only the adapters are trained.

- We trained only **18.5 million parameters (1.18%)** of the model.
- The adapter is a small file (about 70 MB) that can be switched on or off. That is how our chat app can compare "fine-tuned" vs "base" instantly.
- Key settings: rank r = 16, alpha = 32, learning rate 0.0002, one pass over the data (1 epoch).

How do we know if the model improved? (Evaluation)

- **Held-out test sets:** questions the model never saw during training. Testing on training questions would be like grading a student on the exact homework they copied.
- **Accuracy:** percentage of questions answered correctly.
- **Confidence interval (CI):** a range showing how much the score could change by luck. "55% [50–60]" means the true score is very likely between 50% and 60%.
- **p-value (McNemar test):** compares two models on the *same* questions. If $p < 0.05$, the difference is very unlikely to be luck. We call that "significant".

> WHY STATISTICS MATTER
> With 100 test questions, getting 3 more right can happen by chance. Many student projects report "accuracy went up" without checking this. We checked every claim.

CHAPTER 2

What is new in this project (novelty)

Most similar projects follow one recipe: ask ChatGPT to write thousands of Q&A pairs, fine-tune on them, show that a score went up. Our project differs in five ways.

NOVELTY 1 · ANSWERS VERIFIED BY CODE, NOT BY AI

For OS, CN, DBMS and Deep Learning we wrote Python **generators**. Each one creates a random problem and then **runs the real algorithm** to compute the answer: it simulates the CPU schedule, the page-replacement frames, the CRC division, the attribute closure, the convolution. So every one of the 3,000 training answers is **correct by construction**.

AI-written answers can contain mistakes, and a small model copies them. Ours cannot. The generators are themselves tested against textbook examples (Silberschatz) and, for Deep Learning, against **PyTorch**, which computes the same answers independently.

NOVELTY 2 · A CONTROLLED EXPERIMENT (ABLATION) PROVING THE DATA MATTERS

We trained two models that differ in exactly one thing: **FT-B** got the verified problems, **FT-A** did not. FT-B improved on numericals; FT-A did not move from the base model. This shows the improvement comes from the verified data, not just from "more training".

NOVELTY 3 · AI-WRITTEN DATA IS FILTERED BY AGREEMENT WITH OFFICIAL KEYS

For GATE/ISRO past questions, Gemini wrote explanations **without seeing the answer key**. We kept an explanation only if Gemini's answer matched the official key (990 of 1,154 kept). Wrong reasoning was thrown away instead of being learned.

NOVELTY 4 · FAIR AND CAREFUL TESTING

- Test questions are never in training: we removed overlaps with a 13-word matching check, and the generator test sets use different number ranges.
- GATE papers are split by year: up to 2020 for training, and half of 2021–2026 kept only for testing.
- Every score has a confidence interval and a significance test.
- "Format failures" (answer not written in the expected form) are counted separately, so learning the format is not mistaken for learning the subject.

NOVELTY 5 · EVERYTHING ON A LAPTOP, WITH A USABLE PRODUCT

All training and testing ran on an **integrated Intel GPU**, not a cloud server. We solved real engineering problems (memory limits, the GPU not supporting 4-bit models, laptop sleep interrupting training) and delivered a working **chat app** with model switching, a **practice app** that grades the model automatically, and a **research paper**.

In addition, the **error analysis** (Chapter 7) gives a clear finding: the model fails mainly at *executing* procedures, not at knowing them. That points to specific next steps.

CHAPTER 3 · WEEK 1

Step 1: Planning and the baseline

1. **Research.** We read the key papers (LoRA, QLoRA, Qwen2.5, CS-Bench, GuideLM, Self-Instruct, catastrophic forgetting) and surveyed datasets. The plan is in `PROJECT_PLAN.md`.
2. **Chose the base model:** Qwen2.5-1.5B-Instruct, small enough for a laptop and good at following instructions.
3. **Wrote the first generators (OS):** CPU scheduling (FCFS, SJF, SRTF, Round Robin, Priority), page replacement (FIFO, LRU, Optimal), Banker's algorithm, disk scheduling (FCFS, SSTF, SCAN, C-SCAN, LOOK, C-LOOK). Tests check them against textbook worked examples.
4. **Built the evaluation harness** (`eval/run_eval.py`): one program that asks the model every test question, grades the answers and saves everything.
5. **Measured the baseline:** the untrained model, asked directly (B0, "zero-shot") and with 3 worked examples in the prompt (B1, "3-shot").

What a generated problem looks like

```
Question: Disk requests arrive for cylinders 83, 73, 36, 68, 118, 77, 67, 185, 130.
The head starts at 164. Compute the total head movement under FCFS.

Solution (computed by code):
Head path: 164 → 83 → 73 → 36 → 68 → 118 → 77 → 67 → 185 → 130
Total = 81 + 10 + 37 + 32 + 50 + 41 + 10 + 118 + 55 = 434 cylinders
Final answer: 434 cylinders
```

Every answer ends with a **"Final answer:"** line, so a grader program can check it automatically.

Baseline finding

Test	B0 zero-shot	B1 3-shot
CS-Bench (concept MCQs)	51.2%	58.0%
OS numericals	4%	4%
CN numericals	1%	2%
DBMS FD problems	15%	12%
MBPP (Python coding)	41.5%	41.5%

> WHAT THIS MEANS
>
> The base model can answer some concept questions, but it is almost useless on numericals, and showing it 3 examples does not help. This is the gap the project targets.

CHAPTER 4 · WEEK 2

Step 2: Building the dataset

A model is only as good as what it learns from. We combined eight sources, each with its own quality check:

Source	Examples	What it teaches	Quality check
Verified generators (OS, CN, DBMS)	2,400	numerical problem solving	answers computed by code
GATE / ISRO past questions	978	exam-style MCQs with explanations	kept only if the explanation matches the official key
Concept Q&A (Gemini)	1,458	theory across a 133-topic syllabus	MCQ/TF answers must match the key
Open textbooks Q&A	1,170	grounded explanations	answerable from the textbook passage
Spider	1,335	writing SQL from English	the correct SQL must run
MBPP + CodeAlpaca	1,495	Python programming	tests pass / code parses
Replay (general)	499	keeps general ability	length filter

Cleaning steps

1. **Remove near-duplicates** (MinHash), so the model does not memorise repeated questions.
2. **Remove test-set overlap** (13-word matching against every test set). This protects the fairness of the results.
3. **Length filter:** at most 1,024 tokens per example, which fits the laptop's memory.
4. **Split:** 8,868 training and 467 validation examples (2.68 million tokens in total).

WHY GEMINI WAS USED CAREFULLY

Gemini's free tier allows about 20 requests per model per day, so the work was spread over several days and several Gemini models, with caching so nothing was asked twice. Whenever Gemini produced an answer that could be checked against a key, it was checked.

RESPONSIBLE HANDLING

Exam papers (GATE/ISRO) are copyrighted, so their text is used for training but **never uploaded** to the public GitHub repository. Textbooks used are openly licensed (CC BY / CC BY-SA). The API key is stored locally and never committed.

CHAPTER 5 · WEEK 3

Step 3: Training on a laptop

We trained two models with identical settings so that the comparison is fair:

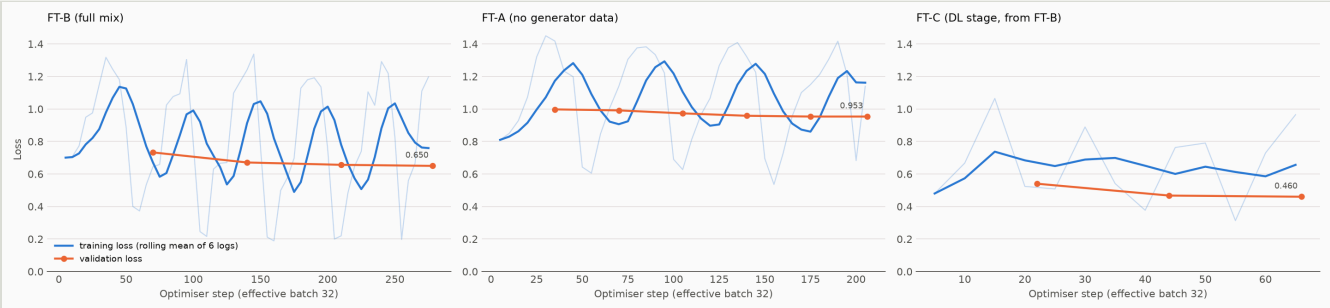
Model	Training data	Examples	Steps	Time
FT-B	full mix	8,868	278	≈ 4.8 h
FT-A	same mix **without** the verified generators	6,588	206	≈ 3.7 h

How training works, simply

1. Take 32 examples (one "step"). Because of memory limits, the laptop processes them one at a time and adds up their effect ("gradient accumulation").
2. For each example, the model predicts the ideal answer word by word. The **loss** measures how wrong those predictions are. We only count loss on the *answer*, not on the question.
3. Adjust the LoRA adapter slightly to reduce the loss. Repeat for all steps.
4. Every 35–70 steps, check the loss on validation examples the model is not trained on, and save a checkpoint.

Engineering problems we solved

Problem	Solution
4-bit QLoRA does not work reliably on Intel GPUs	Used standard LoRA in bf16; the adapter is the same size
Out of memory at batch size 4 (the model's 152,000-word vocabulary makes outputs huge)	Batch size 1 with 32 accumulation steps: same effective batch
Laptop slept / battery died during training	Automatic checkpoints, resume from the last checkpoint, a keep-awake helper
Slow generation during testing	Batched generation (64 questions at once): 2.8 → 144 tokens per second

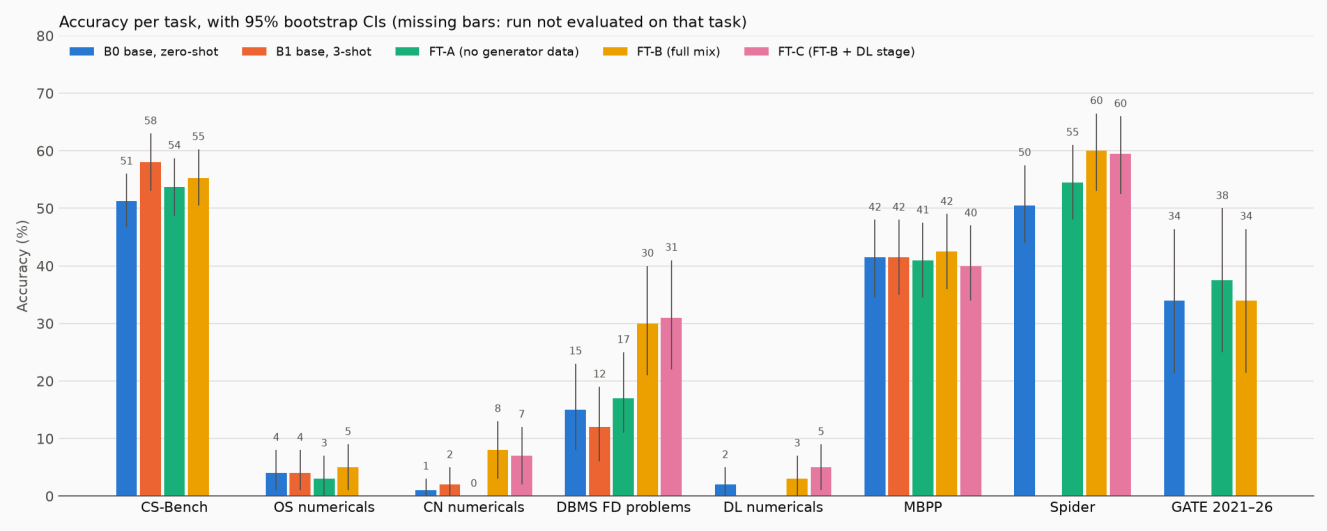


Loss curves. The orange validation loss goes steadily down in every run, which means the model kept learning and did not overfit. The wavy blue line is normal: batches alternate between short answers (low loss) and long explanations (high loss).

CHAPTER 6

Step 4: Results, explained

Test (unseen questions)	Base	FT-A (no verified data)	FT-B (full)	Significant?
Spider: write SQL	50.5%	54.5%	**60.0%**	Yes (p = 0.011)
DBMS functional-dependency problems	15%	17%	**30%**	Yes (p = 0.017)
CN numericals	1%	0%	**8%**	Yes (p = 0.039)
CS-Bench concepts	51.2%	53.8%	55.2%	No
OS numericals	4%	3%	5%	No
MBPP: Python coding	41.5%	41.0%	42.5%	No (kept, not lost)
GATE CS 2021–26	33.9%	37.5%	33.9%	No (only 56 questions)



Every model on every test. The thin black lines are 95% confidence intervals: when two bars' lines overlap a lot, the difference may be luck.

How to read these results

- **Biggest win: database tasks.** FD-closure and key problems went from 3% to 33% within that family. SQL writing improved by 9.5 points.
- **The ablation proves the cause.** FT-A (same training, minus verified problems) stays at base level on numericals. So the verified data is what teaches these procedures.
- **No catastrophic forgetting.** Coding (MBPP) and GATE did not get worse.
- **CS-Bench improved a little** (+4), but half of that came from the model writing its answer in the right format more often. We reported this honestly.

LIMITATION TO MENTION

FT-B trained for 35% more steps than FT-A, so part of the difference could come from extra training. A perfectly matched comparison is listed as future work.

CHAPTER 7 · WEEK 4

Step 5: Error analysis: why the model still fails

Scores tell you *how much*; error analysis tells you *why*. We took 50 random wrong answers from FT-B, read each one and gave it a category.

Error type	Share	Example
Procedure / step tracking	52%	Stops computing an attribute closure after one step; orders FCFS by process number instead of arrival time
Knowledge gap	16%	Confuses deadlock avoidance with deadlock prevention
Schema hallucination (SQL)	12%	Uses a column from the wrong table
Arithmetic / units	8%	Writes 0.96 ms as 9.6 ms
Misread the task	6%	Returns a number when the tests expect True/False
Answer contradicts own reasoning	4%	Explains why A is right, then answers B
Unclear question	2%	Badly translated benchmark question

THE KEY INSIGHT

The model learned the **format and the method** perfectly: zero formatting failures in the sample. What it cannot do reliably is **carry out** a long procedure step by step without losing track. It imitates the shape of a solution without truly executing it. This is a known limit of small models, and our analysis shows exactly where it bites.

What would fix each error type

- Procedure errors → more training, and solutions that restate the full state after each step, with self-checks ("waiting times must be ≥ 0").
- Arithmetic → let the model call Python as a calculator (tool use).
- SQL hallucination → run the query, and retry if the database reports "no such column".
- Knowledge gaps → more concept data for OS/CN, or textbook retrieval.

CHAPTER 8 · EXTENSION

Step 6: Adding Deep Learning (FT-C)

Retraining everything from scratch with DL would take about 8.5 hours. Instead we used **continued training**: start from FT-B's adapter and train a short extra stage.

1. **10 new verified DL generators:** convolution output size and parameters, pooling, MLP parameter count, forward pass, softmax and cross-entropy, gradient descent, backpropagation through a sigmoid neuron, momentum, attention weights, precision/recall/F1. Each one is checked against **PyTorch** in the tests.
2. **DL concept Q&A** from Gemini over 36 DL topics (CNNs, RNNs, LSTMs, Transformers, regularisation, optimisers…).
3. **Replay data:** 1,200 examples from the original mix are mixed in, so the model does not forget DBMS, CN, SQL and coding.
4. **Trained FT-C** on 2,107 examples: 66 steps, **70 minutes**.
5. **Tested only the five most informative tasks** to save about 1.5 hours.

```
Example DL problem: A conv layer receives 30 × 30 × 48. It has 10 filters of 11 × 11,
stride 2, padding 5. Output size = ⌊(30 + 2·5 − 11) / 2⌋ + 1 = 15 → 15 × 15 × 10.
Parameters = (11·11·48 + 1) × 10 = 58,090.
```

Test	Base	FT-B	FT-C	Meaning
DL numericals: share of correct numbers	12.6%	15.8%	**26.0%**	Significant (p = 0.013)
DL numericals: fully correct	2%	3%	5%	Small; arithmetic slips
DBMS FD problems	15%	30%	31%	Kept
CN numericals	1%	8%	7%	Kept
Spider SQL	50.5%	60.0%	59.5%	Kept
MBPP coding	41.5%	42.5%	40.0%	Kept (difference is luck)

WHY TWO DL SCORES?

A DL answer often has several numbers (output height, width, depth, parameters). "Fully correct" needs *all* of them right, so one slip gives zero. "Share of correct numbers" gives partial credit, like an examiner giving step marks. It shows FT-C learned the formulas (convolution 22→47%, softmax 0→56%, metrics 22→44%) even when one multiplication goes wrong.

HONEST NOTE

Gradient-descent and momentum problems got worse, and attention and backpropagation stayed at 0%. These need long chains of decimal arithmetic, which is exactly the weakness found in the error analysis.

CHAPTER 9

Step 7: The chat app, practice demo and paper

Chat app (like ChatGPT/Claude)

- Sidebar with past conversations, saved in your browser.
- Model switch: **FT-C, FT-B, FT-A or the base model**, to show the difference live.
- Answers appear word by word (streaming), with formatted math, tables and highlighted code, plus copy and stop buttons.
- Runs fully on the laptop. Each answer takes about 20–45 seconds.

How it works: a Python server (FastAPI) loads the base model once and attaches all three LoRA adapters by name. Switching models just switches the adapter, with no reloading.

Verified practice demo

- Choose OS, CN, DBMS or Deep Learning.
- The app generates a brand-new problem with a computed answer.
- The model solves it, and the app shows ✅ or ❌ with the correct answer.

Research paper

A 4-page IEEE-style paper (`paper/CSE-Tutor_paper.pdf`) with abstract, method, results tables, figures, error analysis, limitations and 15 references.

How to run everything

Goal	Command (inside the project folder)
Chat with the model	`python demo/server.py` , then open http://127.0.0.1:8000
Practice problems demo	`python demo/app.py` , then open http://127.0.0.1:7860
Run all 44 tests	`python -m unittest discover -s tests -t .`
Rebuild results tables	`python eval/summarize.py`
Rebuild figures	`python eval/plots.py`
Rebuild the paper PDF	`paper\build_pdf.cmd`

BEFORE YOUR REVIEW

The trained adapters are in `outputs/` on the laptop and are **not on GitHub**. Keep a backup. Start the chat app 2 minutes before the demo, because loading the model takes time. A good live question: *"For R(A, B, C, D) with FDs A → B, B → C, C → D, find the closure of A and all candidate keys."*

Project timeline

Phase	Main outputs
Week 1	Plan, OS generators + tests, evaluation harness, baselines B0/B1
Week 2	CN + DBMS generators, GATE/ISRO, Gemini Q&A, textbooks, Spider/MBPP, cleaned dataset
Week 3	Trained FT-B and FT-A, evaluated on 7 tests with statistics
Week 4	Error analysis, figures, Gradio demo, final report
Extension	DL generators, FT-C, chat app, research paper

CHAPTER 10

Likely review questions, with simple answers

1. Why a small model instead of GPT-4?

It runs offline on a student's laptop at no cost and keeps data private. The project asks how far a small model can be pushed with good data.

2. What exactly is your novelty?

Training answers for numerical topics are computed by code, so they are guaranteed correct, and an ablation (FT-A vs FT-B) proves that this verified data causes the numerical gains. Plus fair testing with statistics, all on a laptop.

3. Why LoRA and not full fine-tuning?

Full fine-tuning updates 1.5 billion numbers and does not fit in laptop memory. LoRA trains 18.5 million (1.18%), and the result can be switched on and off as a small adapter file.

4. How do you know the model did not just memorise the test?

The test sets were never trained on. We removed any 13-word overlap, generator test problems use different number ranges, and GATE is split by year.

5. What does $p < 0.05$ mean?

If the two models were really equally good, a difference this large would happen by chance less than 5% of the time. So we trust the difference.

6. Why did CS-Bench barely improve?

It tests broad knowledge, and our data focused on procedures. About half of its +4 points came from the model writing answers in the correct format. 3-shot prompting is still slightly better there.

7. Why is OS scheduling still at 0%?

Long simulations need perfect step tracking. The model copies the Gantt-chart layout but fills in wrong times. The error analysis documents this, and tool use would fix it.

8. How did you add Deep Learning without forgetting?

Continued training from FT-B with 1,200 "replay" examples from the old data mixed in. DBMS, CN, SQL and coding scores stayed the same.

9. Why two different DL metrics?

Exact match needs every number right, which is too harsh to show learning. Value-level credit is like step marks and shows the formulas were learned (26% vs 15.8%, significant).

10. What would you do with more time?

Train more epochs; give the model a calculator (tool use); write solutions that re-check each step; preference training (DPO) using correct/incorrect pairs from the generators; a step-matched ablation; and run MMLU for general knowledge.

11. Where did Gemini help, and how did you stop its mistakes?

Concept Q&A, GATE explanations and textbook Q&A. We only kept answers that matched official keys or could be answered from the textbook passage, and we never used Gemini for the numerical answers.

12. What were the hardest engineering issues?

Intel GPU limits (no 4-bit, memory), slow generation (fixed with batching), and the laptop sleeping during multi-hour training (fixed with checkpoints and resume).

CHAPTER 11

Glossary

Ablation	An experiment that removes one ingredient to see its effect (FT-A = FT-B minus verified data).
Adapter	The small set of LoRA weights added to the frozen model; it can be switched on or off.
Baseline	The model before any training, used for comparison (B0, B1).
Checkpoint	A saved copy of training progress, so training can resume after an interruption.
Confidence interval	The range in which the true score very likely lies.
Contamination	Test questions leaking into training data, which makes scores unfairly high. We removed it.
Epoch	One full pass over the training data.
Fine-tuning	Further training of a pre-trained model on a specific task.
Few-shot / zero-shot	Asking with or without worked examples in the prompt.
Generator	Our Python program that creates a random problem and computes its correct answer.
Hallucination	The model stating something false or invented, e.g. a SQL column that does not exist.
Learning rate	How big each adjustment is during training.
LoRA	Low-Rank Adaptation: trains small extra matrices instead of the whole model.
Loss	A number measuring how wrong the model's predictions are; training lowers it.
McNemar test	A significance test comparing two models on the same questions.
Parameter	One learned number inside the model.
Replay data	Old training examples mixed into new training to prevent forgetting.
SFT	Supervised fine-tuning: learning from (question, ideal answer) pairs.
Token	A piece of a word; models read and write tokens.
Validation set	Examples kept aside during training to check that the model generalises.

Full details: `docs/FINAL_REPORT.md` · `paper/CSE-Tutor_paper.pdf` · weekly reports in `docs/` · results in `eval/results/RESULTS.md`.